6. Hafta

Kalıtım ve Kompozisyon

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- Kalıtımın neden gerektiğini ve is-a ilişkisini açıklayabilecek; `public` kalıtımla bir taban sınıftan türetilmiş sınıf tanımlayıp `protected` üyeleri (türetilmişin erişebildiği, dışarının erişemediği) doğru kullanabileceksiniz;
- Türetilmiş kurucudan taban kurucusunu başlatma listesiyle çağırarak kurucu zincirini kurabilecek, kurucu/yıkıcı çağrı sırasını (taban önce oluşur, en son yıkılır) açıklayabilecek ve nesne dilimlemesi tuzağından kaçınabileceksiniz;
- `virtual` fonksiyonlarla çalışma-zamanı çokbiçimliliğini sağlayabilecek, `override` ile güvenli geçersiz kılma yapabilecek ve taban işaretçiyle güvenli silme için sanal yıkıcı yazabileceksiniz;
- Saf sanal fonksiyon (`= 0`) ile soyut sınıf tanımlayıp bir tipin doğrudan örneklenmesini engelleyebilecek ve türetilmişlere bir sözleşme dayatabileceksiniz;
- Kalıtım (is-a) ile kompozisyon (has-a) arasında yerinde seçim yapabilecek ve bir `Kutuphane`'yi `std::vector<std::unique_ptr<Materyal>>` ile çokbiçimli olarak yönetebileceksiniz.

6.1 Neden Kalıtım? is-a İlişkisi, Taban ve Türetilmiş Sınıf

1 • Ön-öğretim **📣 Neden Önemli?**

Kitaplığınızda üç materyal türü var: basılı kitap, e-kitap ve sesli kitap. Üçünde de bir `baslik`, bir `yazar` ve bir `yil` bilgisi ortak; ama her birinin kendine özgü verisi de var (basılıda sayfa sayısı, e-kitapta dosya boyutu, seslide süre ve seslendiren). Bu üçünü üç ayrı, birbirinden bağımsız sınıf olarak yazarsanız, ortak alanları ve erişimcilerini üç kez tekrarlarsınız; birinde bir düzeltme yapınca ötekileri kolayca unutursunuz. "Ortak olanı bir kez, farklı olanı bir kez" yazmanın bir yolu olmalı — bu ünite tam da o yolu açar.

1 • Ön-öğretim **Ön Bilgi**

Bu üniteye girmeden önce Hafta 3'ten şunları hatırlayın: sınıf, üye fonksiyon, kapsülleme, `public`/`private` erişim belirleyicileri, kurucu ve örtük `this`. Tematik çıkış noktanız Hafta 3'te tanıdığınız somut, bağımsız `kitaplik::Kitap` sınıfıdır; bu hafta *aynı* `kitaplik` ad uzayında ona kardeş bir hiyerarşi eklersiniz (`kitap.hpp` hiç değişmez). Bu ünite `public`/`private` ikilisine üçüncü bir erişim düzeyi (`protected`) ekleyecektir; bu yüzden erişim belirleyicilerinin ne işe yaradığı taze olmalıdır.

Geçen hafta tek bir değer tipini — `Vektor2B`'yi — operatörleriyle donatıp `int` gibi doğal davranan tam bir tip oluşturdunuz. Bu hafta odağınızı tek bir tipten *birden çok ilişkili tipe* genişletiyorsunuz: ortak veriyi ve davranışı bir yerde toplayıp, farklılıkları ayrı ayrı tanımlamayı ele alacaksınız. Çıkış noktanız yine tanıdık bir yer: Kitaplık projesi.

6.1.1 Sorun: üç kez tekrarlanan ortak kısım

Kitaplıkta yönetmek istediğiniz üç materyal türünü düşünün. Kalıtım olmadan bunları yazsaydınız, her birinin başında aynı üç alanı ve aynı erişimcileri tekrar tekrar yazmanız gerekirdi:

```
class BasiliKitap {
   std::string baslik_; // ortak
   Yazar yazar_; // ortak
   int yil_; // ortak
   int sayfaSayisi_; // yalniz basiliya ozgu
   // ... baslik(), yazar(), yil() erisimcileri (ortak) ...
};
class EKitap {
   std::string baslik_; // ortak -- TEKRAR
   Yazar yazar_; // ortak -- TEKRAR
   int yil_; // ortak -- TEKRAR
   double boyutMB_; // yalniz e-kitaba ozgu
   // ... ayni erisimciler yeniden ...
};
```

Aynı üç satırı (ve erişimcileri) her sınıfta yeniden yazmak yalnızca yorucu değildir; asıl tehlike bakımdadır. `yil_` alanının adını değiştirmeye ya da bir erişimciye kontrol eklemeye karar verirseniz, bunu üç yerde birden, hiçbirini atlamadan yapmak zorundasınız. Bu, yazılımda sık uyarılan bir ilkenin — "kendini tekrarlama" (*Don't Repeat Yourself*) — ihlalidir. Ortak kısmı *tek bir yerde* tanımlayıp geri kalan sınıfların onu paylaşmasının bir yolu olmalıdır.

6.1.2 is-a ilişkisi: ne zaman kalıtım?

Bu üç türe dikkatle bakınca aralarında doğal bir ilişki görürsünüz: bir basılı kitap *bir* materyaldir; bir e-kitap *bir* materyaldir; bir sesli kitap *bir* materyaldir. Türkçedeki "...dır/...tir" testini geçen bu ilişki, kalıtımın doğru kullanıldığının ölçütüdür ve İngilizce "is-a" (bir...dır) diye anılır.

Tanım: is-a ilişkisi

Bir tip başka bir tipin *özel bir türü* ise, aralarında *is-a* (*is-a relationship*) ilişkisi vardır: "A bir B'dir" cümlesi doğal ve doğruysa, A tipi B tipinin bir alt türü olarak modellenebilir. "Basılı kitap bir materyaldir" doğru olduğu için `BasiliKitap`, `Materyal`'in bir alt türüdür.

is-a ilişkisi: "A bir B'dir" testini geçen, kalıtımın uygun olduğu ilişki (*is-a relationship*).

Bir tipin bir başkasının özel hâli olduğunu söyleyen bu bağ, **is-a ilişkisi**dir ve ne zaman kalıtım kullanacağınızı belirleyen ilk soruyu verir: iki tip arasında "...dır" demek doğal geliyorsa, orada bir kalıtım fırsatı vardır. Bu testi ciddiye alın; ilerideki ünitelerde "kolay diye" kalıtım kurmanın yol açtığı kırılgan tasarımları göreceksiniz, ve onları hep bu testle yakalayacaksınız.

Analoji

is-a ilişkisini gündelik "taşıt" kavramıyla düşünün. Otomobil, kamyon ve motosikletin üçü de *bir* taşıttır: hepsinde tekerlek, motor ve gaz pedalı ortaktır, ama her biri kendine özgü de davranır. Ortak kısmı bir kez "taşıt" kavramında tanımlar, alt türlerde tekrarlamazsınız; "motosiklet bir taşıttır" cümlesi doğal geldiği için motosiklet, taşıtın bir alt türüdür. Bunu doğrudan koda eşleyin: "taşıt" sizin *taban* tipiniz (`Materyal`), otomobil/kamyon/motosiklet ise ondan türeyen alt tiplerdir (`BasiliKitap`, `EKitap`, `SesliKitap`). Ortak tekerlek/motor, tabandaki ortak alanlara; her taşıtın kendine özgü davranışı ise alt tipin kendi eklerine karşılık gelir.

6.1.3 Taban ve türetilmiş sınıf

Ortak kısmı toplayan sınıfa *taban sınıf*, ondan devralıp özelleşen sınıfa *türetilmiş sınıf* denir. Kitaplık örneğinde `Materyal` tabandır; `BasiliKitap`, `EKitap` ve `SesliKitap` ondan türetilmiş sınıflardır.

Tanım: Taban ve türetilmiş sınıf

Taban sınıf (*base class*), ortak üyeleri tanımlayan ve kendisinden başka sınıfların türetildiği sınıftır. *Türetilmiş sınıf* (*derived class*), bir taban sınıftan devralıp onu özelleştiren, tabanın üyelerine kendi üyelerini ekleyen sınıftır.

taban sınıf: ortak üyeleri tanımlayan, kendisinden türetilen sınıf (*base class*).

türetilmiş sınıf: bir taban sınıftan devralıp özelleştiren sınıf (*derived class*).

Ortak üyeleri tek yerde toplayan sınıfı **taban sınıf** olarak, ondan devralanı ise **türetilmiş sınıf** olarak tanımlarsınız. Türetilmiş sınıf tabanın tüm üyelerini otomatik olarak alır; siz yalnızca *farkı* yazarsınız. Bu, "ortak olanı bir kez, farklı olanı bir kez" hedefinin tam karşılığıdır.

Önce ortak kısmı tutan taban sınıfı kurun. `Materyal`, üç ortak alanı, onları doldurun bir kurucuyu, bir `baslik()` erişimcisini ve materyali ekrana basan bir `yazdir()` üyesini içerir:

```
// Somut taban: yazdir() virtual'dir ama saf sanal DEGILDIR (Materyal m = bk;
// derlenir ve nesne dilimlemesi gozlenebilir kalir).
class Materyal {
public:
    Materyal(std::string baslik, Yazar yazar, int yil);
    const std::string& baslik() const { return baslik_; }
    virtual void yazdir() const;
    virtual ~Materyal();
```

```
39
40 protected:
41    std::string baslik_;
42    Yazar yazar_;
43    int yil_;
44 };
```

Bu bildirimin iki ayrıntısına şimdiden dikkat edin. Birincisi, `yazdir()` bildiriminin başındaki `virtual` anahtar kelimesidir; onu bu ünitede yalnızca *yerine koyuyorsunuz*, işlevini Ünite 3'te "canlandıracaksınız". İkincisi, alanların `private` değil `protected` bölümünde durmasıdır — bunun nedenini birazdan göreceksiniz. Şimdilik yapıyı tamamlayın: ilk türetilmiş sınıfınız `BasiliKitap`, `Materyal`'den türer ve yalnızca kendi ek alanını (`sayfaSayisi_`) ekler.

```
48 class BasiliKitap : public Materyal {
49 public:
50    BasiliKitap(std::string baslik, Yazar yazar, int yil, int sayfaSayisi);
51    void yazdir() const override;
52    ~BasiliKitap() override;
53
54 private:
55    int sayfaSayisi_;
56 };
```

6.1.4 `public` kalıtım ve `: public Materyal` sözdizimi

Türetme, sınıf adından sonra iki nokta ve `public Materyal` yazılarak bildirilir: `class BasiliKitap : public Materyal`. Buradaki `public` sözcüğü kalıtımın *türünü* belirtir ve is-a anlamındaki kalıtımın standart biçimidir.

> **Tanım: `public` kalıtım**
>
> *`public` kalıtım* (*public inheritance*), tabanın `public` arayüzünü türetilmiş sınıfta da `public` olarak koruyan kalıtım biçimidir. is-a ilişkisini ifade eder: türetilmiş nesne, tabanın her `public` üyesiyle tıpkı bir taban nesnesi gibi kullanılabilir.

public kalıtım: tabanın public arayüzünü türetilmişte de public tutan, is-a anlamındaki kalıtım (*public inheritance*).

Tabanın arayüzünü türetilmişte de açık tutan bu biçim, **public kalıtım**tır. Pratikte anlamı şudur: `BasiliKitap`, `Materyal`'in `baslik()` erişimcisini ayrıca yazmasa da ona sahiptir; `bk.baslik()` çağrısı devralınan üye sayesinde doğrudan çalışır. Türetilmiş sınıf tabanın `public` arayüzünün tamamını miras alır ve dışarıya aynı arayüzü sunar.

6.1.5 `protected`: kalıtım hattına açık üyeler

Şimdi az önce ertelediğiniz soruya dönün: `Materyal`'in üç ortak alanı neden `private` değil de `protected` bölümünde? Cevap, türetilmiş sınıfların bu alanlara erişebilmesi gerektiğidir. `private` bir üye *yalnızca* onu tanımlayan sınıfın kendisine açıktır; ondan türeyen sınıflar bile göremez. `public` ise herkese açıktır. `protected` bu ikisinin arasındadır: sınıfın kendisine *ve ondan türeyenlere* açık, ama dışarıya kapalıdır.

> **Tanım: `protected` üye**
>
> *`protected` üye* (*protected member*), sınıfın kendisine ve ondan türeyen sınıflara açık olan, ama sınıf dışındaki koda kapalı olan üyedir. Dışarıdan erişim açısından `private` gibi kapalı, kalıtım hattına karşı ise `public` gibi açıktır.

protected üye: sınıfa ve ondan türeyenlere açık, dışarıya kapalı üye (*protected member*).

Sınıfa ve alt sınıflarına açık, dışarıya kapalı bu üçüncü düzey, **protected üye**dir. `Materyal`'in `baslik_`, `yazar_` ve `yil_` alanlarını `protected` yapmanızın nedeni, `BasiliKitap`'ın kendi `yazdir()`'inin bu alanlara *doğrudan* erişebilmesi gerektiğidir. `BasiliKitap::yazdir`'a bakın:

```
38 void BasiliKitap::yazdir() const {
39    std::cout << "Basili Kitap: \"" << baslik_ << "\" - " << yazar_.tamAd()
40              << " (" << yil_ << "), " << sayfaSayisi_ << " sayfa\n";
41 }
```

Buradaki `baslik_`, `yazar_` ve `yil_`, tabandan devralınan `protected` alanlardır; `BasiliKitap` onlara hiçbir erişimci kullanmadan, tıpkı kendi alanıymış gibi ulaşır. Bu alanlar `private` olsaydı, `BasiliKitap` onları göremez, ancak tabanın bir erişimcisi (`baslik()`) üzerinden *okuyabilirdi*. Taban sürümü, aynı alanların ortak biçimini basar:

```
16 void Materyal::yazdir() const {
17    std::cout << "Materyal: \"" << baslik_ << "\" - " << yazar_.tamAd()
18              << " (" << yil_ << ")\n";
19 }
```

Analoji

Üç erişim düzeyini bir evin dolapları gibi düşünün. `public`, evin kapısında herkese açık bir raftır — sokaktaki herkes uzanabilir. `private`, yalnızca ev sahibinin açtığı kişisel bir çekmecedir; çocuklar bile açamaz. `protected` ise eve ve o evden çıkan çocuklara (türetilmiş sınıflara) açık, ama sokaktaki herkese kapalı ortak bir dolaptır. Koda eşleyin: `Materyal`'in `protected` alanları o ortak dolaptır; `Materyal`'in kendisi (ev) ve ondan türeyen `BasiliKitap` (çocuk) açabilir, ama sınıf dışındaki herhangi bir kod (sokak) açamaz.

Dikkat

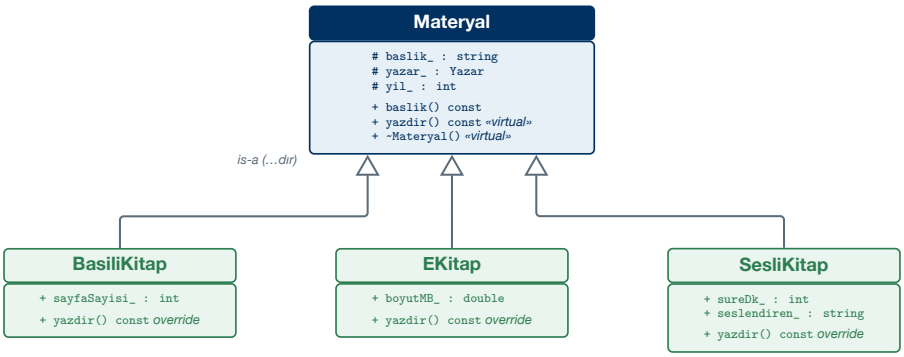
`protected`, dışarıya karşı `private` kadar kapalıdır: sınıf dışındaki bir kod `bk.baslik_` yazamaz (derleme hatası) — yalnızca `bk.baslik()` erişimcisi çalışır. `protected` kapsüllemeyi zayıflatmaz; erişimi yalnızca *kalıtım hattına* genişletir. Alanları gereksiz yere `protected` yapmak yerine, türetilmişin gerçekten doğrudan erişmesi gerekiyorsa `protected`, gerekmiyorsa `private` tutmak iyi bir alışkanlıktır.

Şekil 6.1 bu ünitenin kurduğu hiyerarşiyi bir bakışta özetliyor: en üstte ortak alanları (`protected`) ve `yazdir()` üyesini tutan `Materyal`; altında ondan türeyen üç somut sınıf. Her türetilmiş sınıftan tabana çizilen içi boş üçgen uçlu ok, UML'de is-a (kalıtım) ilişkisini gösterir.

6.1.6 İlk kullanım: devralınan üyeyi çağırmak

Artık ilk türetilmiş nesnenizi oluşturup hem devralınan hem de kendi üyesini kullanabilirsiniz. Bir `BasiliKitap` nesnesi oluşturun; onun üzerinde *devralınan* `baslik()` erişimcisini ve *kendi* `yazdir()`'ini çağırın:

```
10    BasiliKitap bk("Etkili C++", Yazar("Scott Meyers", "ABD"), 2005, 320);
11    std::cout << "Devralinan baslik(): " << bk.baslik() << '\n';
12    bk.yazdir();
```



Şekil 6.1: `Materyal` taban sınıfı ve ondan türeyen üç somut sınıf. Üstte ortak `protected` alanlar ve `virtual yazdir()`; altta her sınıfın kendine özgü alanı. İçi boş üçgen uçlu oklar is-a (kalıtım) ilişkisini gösterir.

`bk.baslik()` çağrısı, `BasiliKitap`'ın hiç yazmadığı bir üyeyi — tabandan devralınan `baslik()`'i — kullanır; `bk.yazdir()` ise `BasiliKitap`'ın kendi sürümünü çağırıp sayfa sayısını da basar. İşte kalıtımın vaadi budur: ortak arayüz (`baslik()`) bir kez tanımlanır, her türetilmiş sınıf onu bedavaya alır; yalnızca farklı olan kısmı (`sayfaSayisi_` ve onu basan `yazdir()`) türetilmişte yazarsınız.

Örnek Olay 6.1: Devralınan arayüz mü, yeni tanım mı?

`EKitap` için `baslik()`'i yeniden yazmanıza gerek var mı? Hayır. `EKitap : public Materyal` yazdığınız an, `baslik()`, `yazar_`, `yil_` ve tüm ortak arayüz `EKitap`'a geçer. Siz yalnızca farkı — `boyutMB_` alanını ve onu basan kendi `yazdir()`'ini — eklersiniz. Üç türetilmiş sınıf da tabana ait `baslik()`'in *aynı* kopyasını paylaşır; `baslik()`'e bir gün kontrol eklemek isterseniz tek bir yeri, `Materyal`'i, değiştirirsiniz ve üçü birden güncellenir. Kalıtım olmadan bu değişikliği üç yerde yapmanız gerekirdi.

Bu ünitede kalıtımın *neden* gerektiğini (ortak kısmı tek yerde toplamak), *ne zaman* uygun olduğunu (is-a testini geçen ilişkiler) ve *nasıl* kurulduğunu (`: public Materyal`, `protected` alanlar) gördünüz. Elinizde artık ortak bir tabandan türeyen ilk somut sınıf var. Gelecek ünitede bu nesnelerin *nasıl kurulduğunu* — taban kısmının önce, türetilmiş kısmın sonra oluşmasını — ve türetilmiş bir nesneyi taban türüne kopyalamanın gizli tuzağını ele alacaksınız.

3 • Katılım Şimdi Siz Deneyin

`BasiliKitap`, `Materyal`'den türetildi ve `Materyal`'in `baslik_` alanı `protected`. (a) `BasiliKitap`'ın `yazdir()`'i, `Materyal`'in `baslik_` alanına doğrudan `baslik_` yazarak erişebilir mi? (b) `baslik_ private` olsaydı yanıt değişir miydi?

Geribildirim. (a) Evet. `baslik_ protected` olduğu için, `Materyal`'den türeyen `BasiliKitap`'ın `yazdir()`'i ona *doğrudan*, hiçbir erişimci kullanmadan ulaşır (yukarıdaki `basilikitap_yazdir`'da tam olarak bunu görürsünüz). (b) Evet, değişirdi: `baslik_ private` olsaydı türetilmiş sınıf ona doğrudan *erişemezdi*; ancak tabanın `public` (ya da `protected`) bir erişimcisi olan `baslik()` üzerinden *okuyabilirdi*. Her iki durumda da sınıf dışındaki bir kod `bk.baslik_`'e doğrudan erişemez — `protected`, dışarıya karşı `private` gibi kapalıdır.

Kendinizi Sınayın 6.1

`Materyal`'in alanlarını `private` yerine `protected` yapmanın türetilmiş sınıflar açısından farkı nedir? Dışarıdan (sınıf dışından) erişim `protected`'te `private`'a göre değişir mi? Yanıtınızda hem kalıtım hattını hem de dışarıyı ayrı ayrı düşünün.

5 • İzleme — Aktarım — Başka Nerede?

is-a hiyerarşileri her yerdedir. Hafta 13'te göreceğiniz istisna türleri bir kalıtım ağacı oluşturur: bir `KitapBulunamadi` hatası *bir* `std::exception`'dır, dolayısıyla ondan türetilir. YG II'de tanışacağınız Qt arayüz sınıfları da aynı örüntüyü kullanır: bir düğme *bir* `QWidget`'tır. "...dır" testini geçen her ilişki bir kalıtım adayıdır; bu tek soruyu her yeni tasarımda sormak, ortak kodu nerede toplayacağınızı doğrudan gösterir. Kitaplık projesinde kurduğunuz `Materyal` tabanı, önümüzdeki üç ünitede bu hiyerarşiyi kurma, davranışını çalışma zamanında seçme ve çokbiçimli bir koleksiyonda tutma adımlarıyla büyüyecek.

6.2 Kurucu Zinciri, Yıkıcı Sırası ve Nesne Dilimlemesi Tuzağı

1 • Ön-öğretim — Neden Önemli?

Bir `BasiliKitap` oluşturduğunuzda içindeki `Materyal` kısmının da doğru kurulması gerekir — ama türetilmiş sınıf, tabanın alanlarını *doğrudan* doldurmamalıdır; bu kapsüllemeyi bozardı. Peki kim önce kurulur, taban mı türetilmiş mi? Yıkımda hangi sıra izlenir? Ve daha sinsisi: bir türetilmiş nesneyi bir taban türündeki değişkene kopyalayınca, çıktının neden "yarısı kaybolur"? Bu ünite bu üç soruyu — kurucu zinciri, yıkıcı sırası ve nesne dilimlemesi — netleştirir.

1 • Ön-öğretim — Ön Bilgi

Bu üniteye girmeden önce Hafta 3'ten kurucu/yıkıcı ve üye başlatma listesini; Hafta 4'ten nesnelerin kurulma ve yıkılma sırasını, kopya kurucuyu ve değer semantiğini hatırlayın. Özellikle Hafta 4'te öğrendiğiniz "kopyalama üye üye gerçekleşir" bilinci, bu ünitenin nesne dilimleme tuzağını açıklarken belirleyici olacaktır. Ünite 1'in taban/türetilmiş yapısı ve `protected` alanları da doğrudan bu ünitenin üzerine kurulur.

Ünite 1'de `Materyal` tabanını ve ondan türeyen `BasiliKitap`'ı tanımladınız; ama nesnenin *nasıl* kurulduğuna henüz bakmadınız. Bir `BasiliKitap` hem tabana ait üç alanı (`baslik_`, `yazar_`, `yil_`) hem de kendi `sayfaSayisi_`'nı içerir. Bu iki parça nasıl ve hangi sırayla dolar?

6.2.1 Kurucu zinciri: taban kurucusunu çağırmak

Türetilmiş sınıfın kurucusu, tabanın `protected` alanlarını elle doldurmaya *kalkışmaz*; bunun yerine tabanın kendi kurucusunu çağırıp o işi ona bırakır. Bu çağrı, başlatma listesinde — diğer üyelerden *önce* — tabanın adıyla yapılır. `BasiliKitap` kurucusuna bakın:

```
29 BasiliKitap::BasiliKitap(std::string baslik, Yazar yazar, int yil,
30                          int sayfaSayisi)
31     : Materyal(std::move(baslik), std::move(yazar), yil),
32       sayfaSayisi_(sayfaSayisi) {
33     std::cout << "BasiliKitap kuruldu\n";
34 }
```

Başlatma listesindeki ilk öğe, tabanın kurucusunu çağıran `Materyal(...)` ifadesidir: bu çağrı, `baslik_`, `yazar_` ve `yil_` alanlarını *tabanın* doldurmasını sağlar. Ancak ondan sonra türetilmiş, kendi alanını (`sayfaSayisi_(sayfaSayisi)`) başlatır. Türetilmiş sınıf tabanın özel işini tabana havale eder; her sınıf yalnızca kendi alanlarından sorumludur. Bu düzene kurucu zinciri denir.

Tanım: Kurucu zinciri

Kurucu zinciri (*constructor chaining*), türetilmiş sınıf kurucusunun, başlatma listesinde önce taban sınıf kurucusunu çağırarak tabana ait alt-nesnenin kurulmasını tabana bırakmasıdır. Türetilmiş kurucu yalnızca kendi ek üyelerini başlatır.

kurucu zinciri: türetilmiş kurucunun başlatma listesinde taban kurucusunu çağırması (*constructor chaining*).

Türetilmiş kurucunun tabanın kurucusunu başlatma listesinde çağırması, **kurucu zinciri**dir. Buradaki en önemli kural şudur: tabanın kurucusunu *açıkça* çağırmazsanız, derleyici sizin adınıza tabanın *varsayılan* (parametresiz) kurucusunu çağırmayı dener. Eğer `Materyal`'in varsayılan bir kurucusu yoksa — ki bizim `Materyal`'imizin yok, çünkü yalnızca üç argüman alan bir kurucusu var — bu bir *derleme hatası* olur. Bu yüzden başlatma listesinde tabanı doğru argümanlarla çağırmak isteğe bağlı değil, zorunludur.

Ortak Hata / Tuzak

Türetilmiş kurucuda tabanın alanlarını doğrudan doldurmaya çalışmak yaygın bir hatadır: `sayfaSayisi_(s), baslik_(b)` yazmak istersiniz, ama `baslik_` tabana aittir ve başlatma listesinde *yalnızca* tabanın kurucusuyla kurulabilir — türetilmiş onu üye gibi başlatamaz (derleme hatası). Doğrusu tabanı çağırmaktır: `: Materyal(b, y, yil), sayfaSayisi_(s)`. Kural nettir: her sınıf yalnızca *kendi* alanlarını başlatır; tabanın alanları için tabanın kurucusunu çağırırsınız.

6.2.2 Kurucu ve yıkıcı sırası

Kurucu zinciri, nesnenin parçalarının hangi sırayla kurulduğunu da belirler. Bir `BasiliKitap` oluşturulurken *önce* taban alt-nesnesi (`Materyal` kısmı) kurulur, *sonra* türetilmiş kısım. Bu mantıklıdır: türetilmiş kısım tabanın üzerine oturur, dolayısıyla altındaki taban önce var olmalıdır. Yıkımda ise tam tersi geçerlidir: önce türetilmiş kısım yıkılır, en son taban.

Tanım: Yıkıcı sırası

Yıkıcı sırası (*destruction order*), bir nesne yok edilirken türetilmiş sınıf yıkıcısının önce, taban sınıf yıkıcısının sonra çalışmasıdır — kurulum sırasının tam tersi. Kurulumda taban önce oluşur, yıkımda taban en son yıkılır.

yıkıcı sırası: yıkımda türetilmişin önce, tabanın sonra çalışması — kurulumun tersi (*destruction order*).

Yıkımın kurulumun tersine işlemesi, **yıkıcı sırası**dır. Bu soyut kuralı somut olarak görmek için, her kurucu ve yıkıcıya bir izleme satırı (`std::cout`) konur. Taban

kurucusu `Materyal kuruldu`, türetilmiş kurucu `BasiliKitap kuruldu` basar; yıkıcılar da benzer izleri bırakır. Şimdi bir `BasiliKitap`'ı bir kapsam bloğu içinde oluşturup o blok bitince otomatik yıkılmasını izleyin:

```
18    std::cout << "-- kapsam basliyor --\n";
19    {
20       BasiliKitap gecici("Temiz Kod", Yazar("Robert C. Martin", "ABD"), 2008,
21                          464);
22    }
23    std::cout << "-- kapsam bitti --\n";
```

Blok içindeki `gecici` nesnesi oluşturulurken izleme çıktısı *önce* `Materyal kuruldu`, *sonra* `BasiliKitap kuruldu` yazar. Blok kapanınca `gecici` kapsam dışına çıkar ve yıkılır; bu kez çıktı *önce* `BasiliKitap yikildi`, *sonra* `Materyal yikildi` olur. Yani:

```
1  -- kapsam basliyor --
2  Materyal kuruldu // taban once
3  BasiliKitap kuruldu // turetilmis sonra
4  BasiliKitap yikildi // turetilmis once (ters sira)
5  Materyal yikildi // taban en son
6  -- kapsam bitti --
```

Analoji

Bu sırayı bir binanın inşası ve yıkımı gibi düşünün. Önce temel (taban) atılır, sonra üst katlar (türetilmiş) örülür — temeli atmadan üst katı öremezsiniz. Yıkım tersine işler: önce üst katlar sökülür, en son temel kaldırılır. Koda eşleyin: `Materyal` alt-nesnesi temeldir, önce kurulur ve en son yıkılır; `BasiliKitap`'a özgü kısım üst kattır, sonra kurulur ve önce yıkılır. İzleme çıktısındaki sıralama tam olarak bu inşaat/yıkım mantığını yansıtır.

6.2.3 Nesne dilimlemesi: yarısı kaybolan nesne

Şimdi bu ünitenin en sinsi tuzağına gelin. Elinizde bir `BasiliKitap` olan `bk` var. Onu bir *taban türü* değişkenine, *değerle* kopyalarsanız ne olur?

```
29    Materyal m = bk; // slicing: yalniz Materyal alt-nesnesi kopyalanir
30    std::cout << "m.yazdir() (dilimlenmis, taban surumu) : ";
31    m.yazdir();
32    std::cout << "bk.yazdir() (turetilmis surumu) : ";
33    bk.yazdir();
```

`Materyal m = bk;` satırı bir `Materyal` *değeri* oluşturur. `m` bir `Materyal`'dir — bir `BasiliKitap` değil — dolayısıyla içinde yalnızca bir `Materyal` kadar yer vardır. `Materyal`'in kopya kurucusu `bk`'nin yalnızca *taban alt-nesnesini* (`baslik_`, `yazar_`, `yil_`) kopyalar; `sayfaSayisi_` ve `BasiliKitap`'a özgü davranış *kesilir*. Sonuç olarak `m.yazdir()`, `BasiliKitap`'ın değil, `Materyal`'in `yazdir()`'ini çağırır ve çıktıda sayfa sayısı *yoktur*. Aynı nesnenin türetilmiş sürümü `bk.yazdir()` ise ek alanı basmaya devam eder. İki çıktının farkı, kaybolan kısmı gösterir.

Tanım: Nesne dilimlemesi

Nesne dilimlemesi (*object slicing*), bir türetilmiş nesneyi taban türüne *değerle* kopyaladığınızda, hedef değişken yalnızca taban alt-nesnesi kadar yer tuttuğu için türetilmişe özgü alanların ve davranışın kesilmesidir. Kalan nesne yalnızca ta-

ban kısmını taşır.

nesne dilimlemesi: türetilmiş nesneyi taban türüne değerle kopyalarken türetilmiş kısmın kesilmesi (*object slicing*).

Türetilmiş bir nesneyi taban türüne değerle kopyalarken türetilmiş kısmın kesilmesi, **nesne dilimlemesi**dir. Bu neden bir tuzaktır? Çünkü hiçbir derleme hatası vermez; kod sessizce derlenir ve çalışır, yalnızca çıktı beklediğinizden "eksik" çıkar. Kaybın kaynağı, `m`'nin türünün kopyalama anında `Materyal`'e sabitlenmiş olmasıdır.

Ortak Hata / Tuzak

Bir türetilmiş nesneyi bir fonksiyona *değerle* (`void yaz(Materyal m)`) geçirmek de sessizce dilimleme yapar — `yaz(bk)` çağrısında `bk`'nin türetilmiş kısmı kesilir ve fonksiyon içinde yalnız taban sürümü görünür. Çözüm, taban türüyle çalışırken *değer* değil *referans* ya da işaretçi kullanmaktır: `void yaz(const Materyal& m)`. Referans/işaretçi nesneyi kopyalamaz, gerçek türünü korur. Bu ayrım, bir sonraki ünitenin (`virtual` ve çalışma-zamanı çokbiçimlilik) temelini oluşturur.

Analoji

Nesne dilimlemesini büyük bir fotoğrafı küçük bir çerçeveye zorla sığdırmaya benzetin. Fotoğrafı olduğundan küçük bir çerçeveye zorlarsanız kenarlar kırpılır; yalnızca ortadaki (ortak) kısım kalır, dışarıdaki ayrıntı — fotoğrafa özgü olan — kesilir. Koda eşleyin: `BasiliKitap` büyük fotoğraftır; `Materyal m` küçük çerçevedir. Kopyalama sırasında yalnızca ortak (taban) kısım çerçeveye sığar; `sayfaSayisi_` gibi türetilmişe özgü ayrıntı çerçevenin dışında kalıp kesilir.

Dilimlemeden nasıl kaçınılır? Türetilmiş nesnelerle çalışırken onları taban türüne *değerle* kopyalamaktan kaçının; bunun yerine taban *referansı* (`Materyal&`) ya da taban *işaretçisi* (`Materyal*`) kullanın. Bir referans ya da işaretçi nesneyi kopyalamaz; sadece var olan nesneye "bakar" ve onun gerçek türünü olduğu gibi korur. İşte bu yüzden bir sonraki ünitede materyalleri hep işaretçi ya da referans üzerinden ele alacaksınız; ve Ünite 4'te `Kutuphane` materyalleri değerle değil, akıllı işaretçilerle tutacaktır. Not olarak: Ünite 3'te göreceğiniz gibi `Materyal`'i *soyut* bir sınıf yapmak bu tuzağı tümden engeller — çünkü değerle örneklenemeyen bir taban zaten dilimlenemez.

Bu ünitede bir nesnenin parçalarının hangi sırayla kurulup yıkıldığını (taban önce kurulur, en son yıkılır) ve türetilmişi taban türüne değerle kopyalamanın türetilmiş kısmı sessizce kestiğini gördünüz. Gelecek ünitede, tam da dilimlemeden kaçınmak için kullandığınız taban işaretçisi/referansı üzerinden "gerçek türe göre davranışın" nasıl sağlandığını — `virtual` fonksiyonları — ele alacaksınız.

3 • Katılım **Şimdi Siz Deneyin**

`BasiliKitap` kurucusunu, taban `Materyal(baslik, yazar, yil)`'i çağıracak biçimde başlatma listesiyle yazdığınızı düşünün. (a) Başlatma listesinde taban kurucusunu hiç çağırmazsanız ne olur? (b) `Materyal m = bk;` satırından sonra `m.yazdir()` ne basar?

Geribildirim. (a) Taban kurucusunu açıkça çağırmazsanız, derleyici tabanın

varsayılan (parametresiz) kurucusunu çağırmayı dener. `Materyal`'in varsayılan kurucusu *yok* (yalnız üç argümanlı kurucusu var); dolayısıyla bir *derleme hatası* alırsınız. Doğrusu: `BasiliKitap(...) : Materyal(baslik, yazar, yil), sayfaSayisi_(s) { }`. (b) `m` yalnızca taban alt-nesnesidir (dilimleme); `m.yazdir()` *taban* sürümünü basar — çıktıda sayfa sayısı yoktur. Türetilmişe özgü kısım kopyalama anında kesilmiştir.

Kendinizi Sınayın 6.2

Bir `BasiliKitap` oluşturulup yok edildiğinde kurucular ve yıkıcılar hangi sırayla çalışır? Ayrıca `Materyal m = bk;` satırında tam olarak ne kaybolur ve `m.yazdir()` neden taban sürümünü basar?

5 • İzleme Aktarım — Başka Nerede?

"Önce taban, sonra türetilmiş kur; tersten yık" örüntüsü katmanlı her tasarımda geçerlidir ve Hafta 4'te öğrendiğiniz RAII sırasının doğrudan devamıdır — kaynaklar oluşturulmanın tersi sırayla serbest bırakılır. Nesne dilimlemesi farkındalığı ise ileride çok pratik bir kararı belirler: bir koleksiyonda nesneleri *değerle mi işaretçiyle mi* tutacaksınız? Ünite 4'te `Kutuphane`'yi tasarlarken tam bu soruyla yüzleşeceksiniz ve dilimlemeyi hatırlamak, `std::vector<Materyal>` yerine neden `std::vector<std::unique_ptr<Materyal>>` seçtiğinizin gerekçesi olacaktır.

6.3 `virtual` Fonksiyonlar, Çalışma-Zamanı Çokbiçimlilik ve Soyut Sınıf

1 • Ön-öğretim Neden Önemli?

Elinizde bir `Materyal*` (ya da `Materyal&`) var, ama işaret ettiği gerçek nesne bir `BasiliKitap`. `p->yazdir()` dediğinizde *hangi* `yazdir()` çalışsın istersiniz — tabanın mı, türetilmişin mi? Bir materyal listesini tek bir döngüde doğru yazdırmak, her elemanın "gerçek türüne göre" davranmasını gerektirir. Ayrıca taban işaretçiyle yanlış silme, sessizce kaynak sızdırabilir. Bu ünite, bu iki sorunu çözen mekanizmayı — `virtual` fonksiyonları — kurar.

1 • Ön-öğretim Ön Bilgi

Bu üniteye girmeden önce Ünite 1–2'nin `Materyal` hiyerarşisini ve taban/türetilmiş `yazdir()`'ini; Hafta 4'ten `std::unique_ptr`'ı (taban işaretçiyle sahiplik ve silme buradan gelir); Ünite 2'nin dilimleme tuzağını ve ondan kaçınmak için işaretçi/referans kullanma gerekçesini hatırlayın. Bir nesnenin "bildirilen" türü ile "gerçek" türü arasındaki ayrım bu ünitede ilk kez belirginleşir.

Ünite 2'de dilimlemeden kaçınmak için türetilmiş nesnelerle taban *işaretçisi* ya da *referansı* üzerinden çalışmanız gerektiğini gördünüz. Şimdi tam da bu işaretçi/referans üzerinden bir üye fonksiyon çağırdığınızda ilginç bir soru doğar.

6.3.1 Statik tip, dinamik tür

Şu satırı düşünün: `Materyal& r = bk;` — burada `r`'nin *bildirilen* türü `Materyal`'dir, ama *gerçekte* bağlı olduğu nesne bir `BasiliKitap`'tır. Bu iki türü ayırt etmek gerekir: derleyicinin kaynağa bakarak gördüğü tür (`Materyal`) ile nesnenin çalışma zamanındaki asıl türü (`BasiliKitap`). `r.yazdir()` çağrısı hangisine göre çözülür? İşte cevabı belirleyen şey `virtual` anahtar kelimesidir.

Tanım: Sanal fonksiyon

Sanal fonksiyon (*virtual function*), tabanda `virtual` anahtar kelimesiyle bildirilen ve çağrısı, işaretçi/referansın bildirilen türüne değil, nesnenin çalışma zamanındaki *gerçek türüne* göre çözülen üye fonksiyondur.

sanal fonksiyon: çağrının nesnenin gerçek (dinamik) türüne göre çözüldüğü, `virtual` bildirimli üye (*virtual function*).

Çağrının nesnenin gerçek türüne göre çözüldüğü üyeyi **sanal fonksiyon** yaparsınız. `Materyal::yazdir`'ı Ünite 1'de `virtual` olarak bildirmiştiniz; işte o bildirim şimdi "canlanıyor". `yazdir() virtual` olduğu için, `r.yazdir()` çağrısı çalışma zamanında `r`'nin bağlı olduğu gerçek nesnenin türüne bakar ve `BasiliKitap::yazdir`'ı çağırır. Eğer `yazdir() virtual` *olmasaydı*, çağrı `r`'nin bildirilen türüne (`Materyal`) sabitlenir ve `Materyal::yazdir` çalışırdı.

Bu davranışı bir işaretçi dizisiyle çokbiçimli olarak görün. Üç farklı türde materyali `Materyal*` olarak tek bir dizide tutup tek döngüde yazdırın:

```
12  BasiliKitap bk("Etkili C++", Yazar("Scott Meyers", "ABD"), 2005, 320);
13  EKitap ek("C++ Temelleri", Yazar("Ayse Yilmaz", "TR"), 2020, 4.5);
14  SesliKitap sk("Kisa Hikayeler", Yazar("Mehmet Demir", "TR"), 2019, 180,
15            "Ali Kaya");
16
17  Materyal& r = bk;
18  std::cout << "Materyal& uzerinden (statik tip Materyal, dinamik tur "
19            "BasiliKitap):\n";
20  r.yazdir();
21
22  std::cout << "\nMateryal* dizisi uzerinden cokbicimli yazdirma:\n";
23  Materyal* dizi[] = {&bk, &ek, &sk};
24  for (Materyal* p : dizi) {
25      p->yazdir();
26  }
```

Dizinin her elemanı bir `Materyal*`'tır; ama `p->yazdir()` çağrısı her seferinde *gerçek* türe göre çözülür: birincide `BasiliKitap::yazdir`, ikincide `EKitap::yazdir`, üçüncüde `SesliKitap::yazdir`. Statik tip üçünde de aynı (`Materyal`) olmasına rağmen, çıktı her materyalin kendi biçiminde gelir. Bu, tek bir arayüz (`yazdir()`) üzerinden farklı türlerin farklı davranmasıdır.

Tanım: Çalışma-zamanı çokbiçimlilik

Çalışma-zamanı çokbiçimlilik (*runtime polymorphism*), aynı taban arayüzü üzerinden çağrılan bir sanal fonksiyonun, nesnenin gerçek türüne göre farklı davranması olgusudur. Sanal çağrının gerçek türe yönlendirilmesine *dinamik gönderim* (*dynamic dispatch*) denir.

çalışma-zamanı çokbiçimlilik: aynı taban arayüzü üzerinden farklı türlerin farklı davranması (*runtime polymorphism*).

dinamik gönderim: sanal çağrının çalışma zamanında gerçek türe

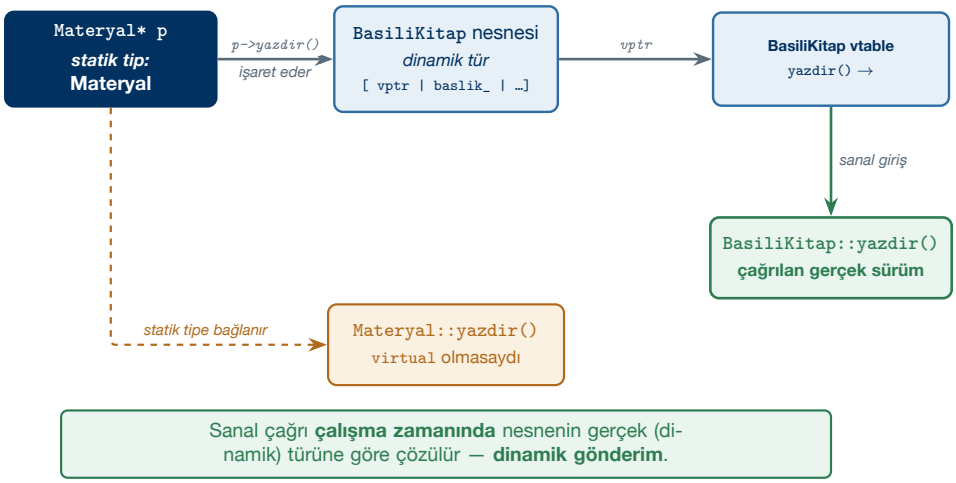
Aynı arayüz üzerinden farklı türlerin farklı davranması **çalışma-zamanı çokbiçimlilik**tir; sanal çağrının gerçek türe yönlendirilmesine ise **dinamik gönderim** denir. Kavramsal olarak, çokbiçimli her nesne gizli bir tür işaretçisi taşır (bu yapı

"sanal fonksiyon tablosu", kısaca vtable fikridir); `virtual` bir çağrı bu tablo üzerinden gerçek türün fonksiyonuna gider. Ayrıntısına girmeden, fikir düzeyinde bunu bilmeniz yeterlidir.

Analoji

Sanal gönderimi bir uzaktan kumandanın "aç" tuşu gibi düşünün. Aynı tuş, televizyona doğrultulduğunda ekranı açar; klimaya doğrultulduğunda soğutmayı başlatır. Tuş (arayüz, yani `yazdir()`) aynıdır; ama yaptığı iş, karşıdaki gerçek cihaza (nesnenin gerçek türüne) göre değişir. Koda eşleyin: `p->yazdir()` tuşa basmaktır; `p`'nin bildirilen türü hangi tuşa bastığınızı değil, *hangi cihaza doğrult-tuğunuzu* belirlemez — davranışı belirleyen, `p`'nin gösterdiği gerçek nesnenin türüdür.

Şekil 6.2, `p->yazdir()` çağrısının nasıl çözüldüğünü gösteriyor: `Materyal* p` işaretçisi (statik tip `Materyal`) bellekteki gerçek `BasiliKitap` nesnesine bakar; nesnenin gizli tür tablosu üzerinden çağrı `BasiliKitap::yazdir`'a yönlenir. Soluk/kesikli karşıt ok ise, `yazdir() virtual` olmasaydı çağrının tabanın `yazdir`'ına bağlanacağını gösteriyor.



Şekil 6.2: `virtual` çağrının çözümü: `Materyal* p` (statik tip `Materyal`), gerçek `BasiliKitap` nesnesinin tür tablosu üzerinden `BasiliKitap::yazdir`'a yönlenir. Kesikli karşıt ok, `virtual` olmasaydı çağrının tabana bağlanacağını gösterir.

6.3.2 `override`: güvenli geçersiz kılma

Türetilmiş sınıf, tabanın sanal fonksiyonunu *yeniden tanımlar*; buna geçersiz kılma denir. `BasiliKitap::yazdir`, `Materyal::yazdir`'ı geçersiz kılar. C++'ta bunu bildiriminin sonuna `override` yazarak işaretlersiniz: `void yazdir() const override;`.

Tanım: Geçersiz kılma ve `override`

Geçersiz kılma (*overriding*), türetilmiş sınıfın tabanın bir sanal fonksiyonunu aynı imzayla yeniden tanımlamasıdır. `override` anahtar kelimesi bu niyeti derleyiciye bildirir: imza tabandaki sanal fonksiyonla eşleşmezse derleyici *hata* verir.

geçersiz kılma: türetilmişin taban sanal fonksiyonunu yeniden tanımlaması (*overriding*).

Türetilmişin tabanın sanal üyesini yeniden tanımlaması **geçersiz kılma**dır; `override` bunu güvenli kılar. Neden gerekli? `override` yazmasaydınız ve imzayı yanlış yazsaydınız — diyelim son `const`'u unutup `void yazdir() override` yerine `void yazdir()` yazsaydınız — derleyici bunu geçersiz kılma *değil*, yepyeni ayrı bir fonksiyon sanır ve sessizce kabul ederdi. Sonuç: `p->yazdir()` sizin sürümünüzü hiç çağırmaz, sessizce tabanınkine gider. `override`, imzalar eşleşmediğinde derleme hatası vererek bu sessiz hatayı yakalar; bu yüzden her geçersiz kılmada onu yazmak standart bir alışkanlıktır.

Ortak Hata / Tuzak

İmza uyuşmazlığı en sık `const`'u unutmakta görülür: taban `void yazdir() const` iken türetilmişte `void yazdir()` (const'suz) yazmak, iki *ayrı* fonksiyon oluşturur — geçersiz kılma gerçekleşmez ve çokbiçimli çağrı tabana gider. `override` olmadan bu sessizce derlenir; `override` ile derleyici "eşleşen sanal fonksiyon yok" diye durur. Alışkanlık nettir: geçersiz kıldığınız her üyeye `override` ekleyin.

6.3.3 Sanal yıkıcı: taban işaretçiyle güvenli silme

Sanal fonksiyonların en kritik uygulamalarından biri yıkıcıdır. Bir türetilmiş nesneyi bir taban işaretçisi üzerinden silmeyi düşünün: `Materyal* p = new BasiliKitap(...); delete p;`. Burada `delete p` hangi yıkıcıyı çağırır? Eğer `~Materyal() virtual` *değilse*, yalnızca taban yıkıcısı (`~Materyal`) çalışır; türetilmişin yıkıcısı (`~BasiliKitap`) hiç çağrılmaz. Türetilmişin ayırdığı ek kaynaklar temizlenmeden kalır — bu tanımsız davranıştır (*undefined behavior*).

Tanım: Sanal yıkıcı

Sanal yıkıcı (*virtual destructor*), tabanda `virtual` bildirilen yıkıcıdır. Bir türetilmiş nesne taban işaretçisi üzerinden silindiğinde, önce türetilmiş yıkıcının, sonra taban yıkıcısının çalışmasını — yani doğru yıkım zincirini — güvenceye alır.

sanal yıkıcı: taban işaretçiyle silmede türetilmiş yıkıcının da çalışmasını sağlayan `virtual` yıkıcı (*virtual destructor*).

Taban işaretçiyle silmede türetilmişin de temizlenmesini sağlayan yıkıcı, **sanal yıkıcı**dır. Bu yüzden `Materyal`'in yıkıcısını Ünite 1'de `virtual ~Materyal();` diye bildirmiştiniz. Kural genel ve kesindir: taban işaretçisiyle silinebilecek her polimorfik tabanın yıkıcısı `virtual` olmalıdır. Bunu `std::unique_ptr<Materyal>` ile birlikte görün — akıllı işaretçi kapsam çıkışında `delete` çağırır ve sanal yıkıcı sayesinde doğru zincir işler:

```
32    std::cout << "-- unique_ptr<Materyal> olusturuluyor --\n";
33    std::unique_ptr<Materyal> p = std::make_unique<BasiliKitap>(
34       "Tasarim Desenleri", Yazar("Erich Gamma", "ABD"), 1994, 395);
35    std::cout << "-- unique_ptr sifirlaniyor (yikim tetiklenir) --\n";
36    p.reset();
37    std::cout << "-- yikim tamamlandi --\n";
```

`p.reset()` çağrıldığında `unique_ptr` sahip olduğu nesneyi siler; `~Materyal() virtual` olduğu için çıktı önce `BasiliKitap yikildi`, sonra `Materyal yikildi` olur (Ünite 2'nin yıkıcı sırasıyla tutarlı). Yıkıcı sanal olmasaydı yalnızca `Materyal yikildi` basılır, türetilmiş kısım hiç temizlenmezdi.

Analoji

Sanal yıkıcıyı bir kargonun doğru sorumluya teslimi gibi düşünün. Bir apartman dairesine kargo geldiğinde, kapıda yalnızca binanın genel görevlisi (taban yıkıcı) imza atarsa, dairenin kendi özel eşyası (türetilmişin ek kaynağı) toplanmadan kalır. `virtual` yıkıcı ise doğru daireyi (türetilmiş) sorumluyu da çağırır; önce daire, sonra bina düzeyinde her şey sırayla toplanır. Koda eşleyin: `delete p` kapıdaki tesliminizdir; `virtual` yıkıcı olmadan yalnızca taban "imza atar", türetilmişin kaynağı ortada kalır.

6.3.4 Saf sanal fonksiyon ve soyut sınıf

Bazen tabanın bir üyesi için makul bir ortak gövde *yoktur*; her türetilmişin onu kendi biçiminde tanımlaması *zorunludur*. Böyle bir üyeyi tabanda gövdesiz, `= 0` ekiyle bildirirsiniz. Bunu, kitaplıktan bağımsız, gündelik bir örnekle — `Sekil` ve alt türleri — görün: her şeklin bir *alanı* vardır, ama "genel bir şeklin" alanını hesaplamanın ortak bir yolu yoktur; alan yalnızca somut bir şekil (daire, kare) için tanımlıdır.

```
7  class Sekil {
8  public:
9     virtual double alan() const = 0;
10    virtual ~Sekil() = default;
11 };
```

`virtual double alan() const = 0;` bildirimi, `Sekil`'in bu işlevi *tanımlamadığını*, her türetilmişin tanımlamak *zorunda* olduğunu söyler. Sondaki `= 0` bunu bir saf sanal fonksiyon yapar.

Tanım: Saf sanal fonksiyon ve soyut sınıf

Saf sanal fonksiyon (*pure virtual function*), tabanda gövdesiz, `= 0` ekiyle bildirilen ve her türetilmişte tanımlanması zorunlu olan sanal fonksiyondur. En az bir saf sanalı olan sınıf *soyut sınıf* (*abstract class*) olur: doğrudan örneklenemez, yalnızca türetilmişleri örneklenebilir.

saf sanal fonksiyon: tabanda gövdesiz (`= 0`), türetilmişe uygulamayı zorunlu kılan sanal fonksiyon (*pure virtual function*).

soyut sınıf: en az bir saf sanalı olan, doğrudan örneklenemeyen sınıf (*abstract class*).

Tabanda gövdesiz bırakılan üye bir **saf sanal fonksiyon**dur; en az bir saf sanalı olan sınıf bir **soyut sınıf**tır. Soyut bir sınıfı doğrudan örnekleyemezsiniz: `Sekil s;` bir derleme hatasıdır — "genel şekil" diye örneklenebilir bir nesne yoktur. Yalnızca somut türetilmişleri (`Daire`, `Kare`) örneklersiniz; her biri saf sanal `alan()`'ı kendi biçiminde tanımlamak *zorundadır*, aksi hâlde onlar da soyut kalır. Kullanımını görün:

```
43    // Sekil gov; // DERLEME HATASI: soyut sinif (saf sanal alan()) dogrudan
44    // // orneklenemez.
45    Daire d(3.0);
46    Kare kr(4.0);
47    Sekil* sekiller[] = {&d, &kr};
48    for (Sekil* s : sekiller) {
49       std::cout << "alan = " << s->alan() << '\n';
50    }
```

İlk satır yorumdadır çünkü `Sekil` doğrudan örneklenemez; onun yerine `Daire` ve `Kare` örneklenir ve `Sekil*` üzerinden `alan()` çokbiçimli olarak çağrılır — her şekil kendi alanını hesaplar.

Analoji

Soyut sınıfı salt "taşıt" kavramı gibi düşünün. "Taşıt" tek başına yolda gidemez; o soyut bir fikirdir. Yola ancak somut bir otomobil ya da kamyon çıkar. Aynı biçimde bir soyut sınıf örneklenmez; yalnızca onun somut türetilmişleri nesne olur. Koda eşleyin: `Sekil` "taşıt" gibi soyut bir kavramdır (`Sekil s;` yasaktır); `Daire` ve `Kare` ise yola çıkabilen somut taşıtlardır.

İpucu

`Materyal`'i de `yazdir()`'i saf sanal (`= 0`) yaparak soyutlaştırabilirdiniz; bu hem her materyalin kendi yazdırmasını *zorunlu* kılar hem de Ünite 2'nin dilimleme tuzağını imkânsız hâle getirir (soyut bir taban değerle örneklenemez, dolayısıyla dilimlenemez). Bu hafta `Materyal`'i bilinçli olarak somut tuttunuz ki dilimlemeyi gözlemleyebilesiniz; `Materyal`'in tam soyutlaştırılması ve zenginleştirilmiş çokbiçimli koleksiyon Hafta 7'nin (Çokbiçimlilik) konusudur.

Bu ünitede `virtual` ile çağrının gerçek türe göre çözüldüğünü (dinamik gönderim), `override` ile geçersiz kılmanın güvenli yapıldığını, sanal yıkıcıyla taban işaretçiyle silmenin doğru zinciri işlettiğini ve saf sanal ile soyut sınıf tanımlayarak türetilmişlere bir sözleşme dayatabileceğinizi gördünüz. Gelecek ünitede bu çokbiçimli davranışı bir `Kutuphane` koleksiyonunda kullanacak ve kalıtımı kompozisyondan ayırt etmeyi öğreneceksiniz.

3 • Katılım **Şimdi Siz Deneyin**

`Materyal* p = new BasiliKitap(...);` sonra `p->yazdir();` yazdınız. (a) `yazdir()` `virtual` ise hangi sürüm çalışır? (b) `virtual` olmasaydı? (c) `~Materyal()` `virtual` değilken `delete p;` neden tehlikelidir?

Geribildirim. (a) Dinamik gönderim gereği nesnenin gerçek türü çalışır: `BasiliKitap::yazdir`. (b) `virtual` olmasaydı çağrı bildirilen tipe (`Materyal`) bağlanırdı: `Materyal::yazdir`. (c) Yalnızca taban yıkıcısı (`~Materyal`) çalışır; türetilmişin (`~BasiliKitap`) ek kaynağı temizlenmez — tanımsız davranış. Bu yüzden taban işaretçiyle silinebilecek her polimorfik tabanın yıkıcısı `virtual` olmalıdır.

Kendinizi Sınayın 6.3

`~Materyal()` `virtual` DEĞİLKEN bir `Materyal* p` bir `BasiliKitap`'a işaret ederken `delete p;` yaparsanız ne olur ve neden? Doğru davranışı sağlamak için ne yapmak gerekir?

5 • İzleme **Aktarım — Başka Nerede?**

Sanal gönderim, Hafta 7'nin (Çokbiçimlilik) çekirdeğidir: `Materyal`'i soyut yapıp sanal bir `ozet()` ekleyerek zenginleştirilmiş bir `Materyal*` koleksiyonu kuracaksınız. YG II'de Qt olay işleyicileri de (`virtual` fonksiyonlar) ve sinyal/yuva mekanizması, aynı çalışma-zamanı çokbiçimliliğine dayanır. Soyut sınıf fikri ise bir "sözleşme" ya da "arayüz" tanımlamanın temelidir: bir tipin ne

yapması gerektiğini (saf sanal üyeler), nasıl yaptığını söylemeden dayatırsınız — bu, büyük sistemlerde parçaları birbirinden bağımsız geliştirmenin anahtarıdır.

6.4 Kompozisyon (has-a) ile Kalıtım (is-a): Ne Zaman Hangisi?

1 • Ön-öğretim — Neden Önemli?

`Kutuphane`'yi tasarlarken kolay bir tuzak var: "kütüphane bir materyal koleksiyonudur, öyleyse `std::vector`'dan türeteyim" demek. Ama bir `Kutuphane` bir vektör *değildir* — bir vektöre *sahiptir*. "Bir...dır" mı yoksa "...e sahiptir" mi sorusunu yanlış yanıtlamak, kırılgan ve sızdıran bir tasarım oluşturur. Bu ünite bu kararı netleştirir ve materyalleri tek bir kapta çokbiçimli olarak tutmayı gösterir.

1 • Ön-öğretim — Ön Bilgi

Bu üniteye girmeden önce Ünite 1–3'ün tüm `Materyal` hiyerarşisini ve sanal gönderimini; Hafta 4'ten `std::unique_ptr` ile `std::vector`'ü ve RAII sahipliğini; Ünite 2'nin dilimleme farkındalığını (materyalleri neden değerle tutmadığınızın gerekçesi) hatırlayın. Bu ünite, önceki üç ünitenin tüm parçalarını tek bir çokbiçimli koleksiyonda birleştirir.

Önceki üç ünitede kalıtımı — ortak kısmı bir tabanda toplayıp is-a ilişkisiyle türetmeyi — her yönüyle kurdunuz. Ama tiplerin birbiriyle kurabileceği tek ilişki kalıtım değildir. Sık gereken bir başka ilişki, bir tipin başka bir tipi *içermesidir*.

6.4.1 has-a ilişkisi ve kompozisyon

Bir `Kutuphane`'nin materyalleri, bir `Materyal`'in de bir yazarı vardır. Ne `Kutuphane` *bir* `Materyal`'dir, ne de `Materyal` *bir* `Yazar`'dır; onun yerine `Kutuphane` materyallere *sahiptir*, `Materyal` bir `Yazar`'a *sahiptir*. Türkçedeki "...e sahiptir / ...i içerir" testini geçen bu ilişki, İngilizce "has-a" diye anılır ve onu koda dökmenin yolu kompozisyondur.

Tanım: Kompozisyon ve has-a ilişkisi

Kompozisyon (*composition*), bir tipin başka bir tipi *üye olarak içermesidir*. Aralarındaki ilişki *has-a* (*has-a relationship*) diye anılır: "A, bir B'ye sahiptir / B içerir" cümlesi doğalsa, A tipi B'yi bir üye olarak barındırır — ondan türemez.

kompozisyon: bir tipin başka bir tipi üye olarak içermesi (*composition*).

Bir tipin başka bir tipi üye olarak barındırması **kompozisyon**dur; bu ilişkinin testi ise **has-a ilişkisi**dir. Kitaplık hiyerarşisinde iki has-a örneği vardır. Birincisi küçük bir bileşen sınıftır: `Materyal` bir `Yazar` üyesi tutar.

has-a ilişkisi: "A bir B'ye sahiptir/içerir" testini geçen, kompozisyonun uygun olduğu ilişki (*has-a relationship*).

```
14 // has-a: Materyal bir Yazar DEGILDIR, bir Yazar'a sahiptir (kompozisyon).
15 class Yazar {
16 public:
17     Yazar(std::string ad, std::string ulke)
18         : ad_(std::move(ad)), ulke_(std::move(ulke)) {}
```

```
19
20      const std::string& ad() const { return ad_; }
21      const std::string& ulke() const { return ulke_; }
22      std::string tamAd() const { return ad_ + " (" + ulke_ + ")"; }
23
24  private:
25      std::string ad_;
26      std::string ulke_;
27  };
```

`Materyal` bir `Yazar`'dan *türemez* ("materyal bir yazardır" saçmadır); onun yerine bir `Yazar yazar_` *üyesi içerir* — bu bir has-a ilişkisidir. İkinci ve daha önemli has-a örneği `Kutuphane`'dir: materyallere sahiptir, dolayısıyla onları bir üye kapta tutar.

```
86  // has-a: Kutuphane bir Materyal DEGILDIR, materyallere sahiptir (kompozisyon
        ).
87  class Kutuphane {
88  public:
89      void ekle(std::unique_ptr<Materyal> m);
90      void hepsiniYazdir() const;
91
92  private:
93      std::vector<std::unique_ptr<Materyal>> raf_;
94  };
```

`Kutuphane`, materyalleri bir `std::vector<std::unique_ptr<Materyal>>` üyesinde (`raf_`) *içerir*. Bir vektörden *türemez*; bir vektöre *sahiptir*.

Analoji

Kompozisyonu bir arabanın motora sahip olması gibi düşünün. Araba bir motora *sahiptir*, ama araba bir motor *değildir*. Aynı biçimde bir bilgisayar bir işlemciye sahiptir; bir kütüphane raflara ve içindeki kitaplara sahiptir. "...e sahiptir" testi has-a'dır ve kompozisyona işaret eder. Koda eşleyin: `Kutuphane` arabadır, `raf_` (materyal kabı) ise arabanın sahip olduğu motordur — araba motoru *içerir*, ondan türemez.

6.4.2 Seçim ölçütü: is-a mı, has-a mı?

Artık iki tip arasındaki ilişkiyi modellerken elinizde net bir ölçüt var. İki cümleyi deneyin: "A bir B'dir" ve "A, B'ye sahiptir". Hangisi doğal geliyorsa, tasarım kararınızı o belirler.

- "A bir B'dir" doğal geliyorsa: **kalıtım** (is-a). Örnek: "basılı kitap bir materyaldir" → `BasiliKitap : public Materyal`.
- "A, B'ye sahiptir" doğal geliyorsa: **kompozisyon** (has-a). Örnek: "kütüphane materyallere sahiptir" → `Kutuphane` bir `vector<...>` üyesi tutar.

Örnek Olay 6.2: Kutuphane'yi std::vector'dan türetmek yanlış

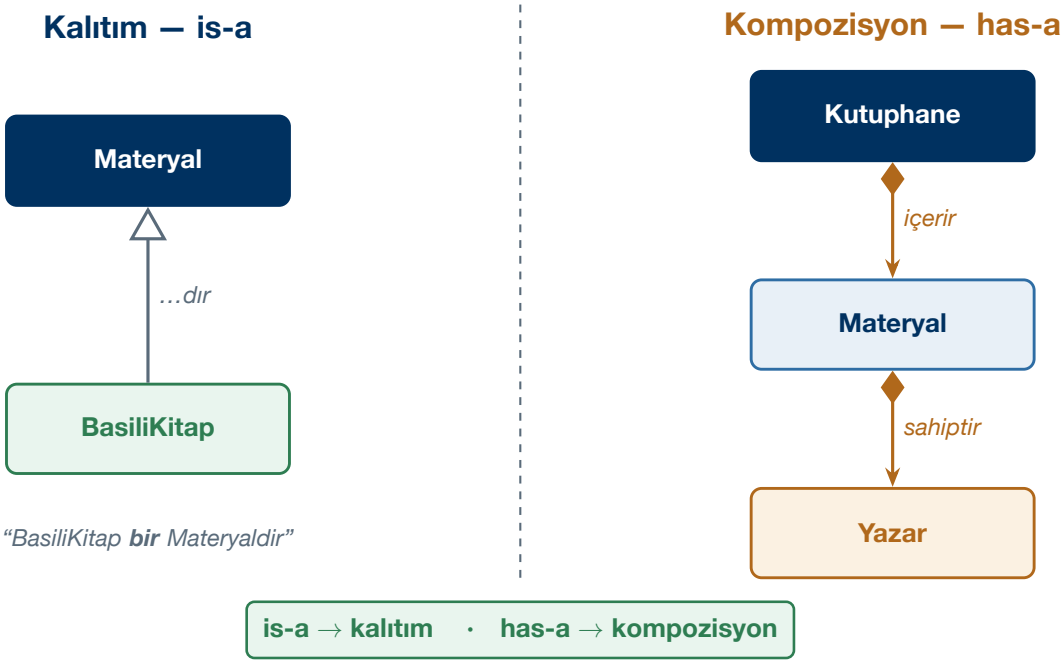
Kolay bir yol gibi görünür: "`Kutuphane`, materyal koleksiyonudur; `std::vector<std::unique_ptr<Materyal>>`'dan türeteyim, `push_back` gibi tüm üyeleri bedavaya alayım." Testi uygulayın: "`Kutuphane` *bir* vektör müdür?" Hayır — bir `Kutuphane` bir vektör *değildir*, bir vektöre *sahiptir*. Kalıtımı zorlarsanız, `Kutuphane`'niz dışarıya vektörün tüm arayüzünü (`clear`,

`resize`, `operator[]` ...) sızdırır; kullananlar `raf`'ı sizin kurallarınız dışında değiştirebilir ve tasarımınız kırılganlaşır. Doğrusu kompozisyondur: vektörü bir `private` üye yapıp yalnızca istediğiniz arayüzü (`ekle`, `hepsiniYazdir`) dışarıya açarsınız.

İpucu

Kararsız kaldığınızda genel öğüt şudur: **belirsizse kompozisyonu tercih edin**. Kompozisyon daha gevşek bir bağ oluşturur — içerdiğiniz tipin yalnızca ihtiyacınız olan kısmını kullanır, geri kalanını dışarı sızdırmazsınız — ve tasarımı daha esnek tutar. Kalıtım güçlü ama sıkı bir bağdır; onu yalnızca is-a testini gerçekten geçen ilişkiler için saklayın.

Şekil 6.3 iki ilişkiyi yan yana koyuyor: solda `BasiliKitap`'tan `Materyal`'e is-a (içi boş üçgen uçlu kalıtım oku, "...dır"); sağda `Kutuphane`'den `Materyal`'e ve `Materyal`'den `Yazar`'a has-a (içi dolu baklava uçlu kompozisyon oku, "...e sahiptir").



Şekil 6.3: Solda is-a (kalıtım): `BasiliKitap` bir `Materyal`'dir — içi boş üçgen uçlu ok. Sağda has-a (kompozisyon): `Kutuphane` materyallere, `Materyal` bir `Yazar`'a sahiptir — içi dolu baklava uçlu ok. is-a → kalıtım, has-a → kompozisyon.

6.4.3 Çokbiçimli koleksiyon: tek kapta üç tür

Şimdi önceki üç ünitenin tüm parçaları birleşiyor. `Kutuphane`, materyalleri `std::unique_ptr<Materyal>` olarak tutar; `ekle` sahipliği kaba devreder, `hepsiniYazdir` ise tek bir döngüde her materyali yazdırır:

```
88 void Kutuphane::ekle(std::unique_ptr<Materyal> m) {
89     raf_.push_back(std::move(m));
90 }
```

```
94 void Kutuphane::hepsiniYazdir() const {
95    for (const auto& m : raf_) {
96       m->yazdir();
97    }
98 }
```

`hepsiniYazdir`, `raf_` üzerinde tek bir döngüyle gezip her eleman için `m->yazdir()`'i çağırır. Kap içindeki her `m` bir `unique_ptr<Materyal>`'dir, ama işaret ettiği gerçek nesne bir `BasiliKitap`, `EKitap` ya da `SesliKitap`'tır. Ünite 3'ün sanal gönderimi sayesinde `m->yazdir()` her seferinde *gerçek türe* göre çözülür; farklı türler, tek kapta, tek döngüde doğru davranır. Kullanımı görün:

```
11    Kutuphane k;
12    k.ekle(std::make_unique<BasiliKitap>(
13       "Etkili C++", Yazar("Scott Meyers", "ABD"), 2005, 320));
14    k.ekle(std::make_unique<EKitap>("C++ Temelleri",
15                            Yazar("Ayse Yilmaz", "TR"), 2020, 4.5));
16    k.ekle(std::make_unique<SesliKitap>("Kisa Hikayeler",
17                               Yazar("Mehmet Demir", "TR"), 2019, 180,
18                               "Ali Kaya"));
19
20    k.hepsiniYazdir();
```

Üç farklı türde materyali aynı `Kutuphane`'ye eklersiniz; `hepsiniYazdir()` her birini kendi biçiminde — basılı kitabı sayfa sayısıyla, e-kitabı boyutuyla, sesli kitabı süresi ve seslendireniyle — yazdırır. Kodda hiçbir yerde "bu bir basılı kitap mı?" diye tür kontrolü yoktur; çokbiçimlilik bu ayrımı sizin yerinize, çalışma zamanında yapar.

Analoji

Çokbiçimli koleksiyonu bir meyve sepeti gibi düşünün. Bir sepet elma, muz ve portakalı bir arada tutar; hepsi *bir* meyvedir. Sepeti tek tek gezip her birini "tartarsınız" — aynı işlem (tartma), her meyvede kendi türüne göre bir sonuç verir. Koda eşleyin: `Kutuphane` sepettir, `raf_` sepetin içidir; `hepsiniYazdir`'ın döngüsü tek tek gezmedir; her `m->yazdir()` ise o meyveyi kendi türüne göre "tartmadır".

6.4.4 Neden değerle değil, işaretçiyle?

Son bir soru kalıyor: `Kutuphane` neden `std::vector<Materyal>` (değerle) değil de `std::vector<std::unique_ptr<Materyal>>` (işaretçiyle) kullanıyor? İki gerekçe vardır ve ikisi de bu haftanın önceki ünitelerinden gelir. Birincisi dilimlemedir: `std::vector<Materyal>`, her elemanı bir `Materyal` *değeri* olarak saklar; bir `BasiliKitap` eklediğinizde Ünite 2'deki gibi dilimlenir — türetilmiş kısmı kesilir, dinamik türü kaybolur ve `yazdir()` artık taban sürümünü çağırır. İşaretçi ise nesneyi kopyalamaz; gerçek türünü olduğu gibi korur, böylece çokbiçimli çağrı çalışır. İkincisi sahipliktir: `unique_ptr`, işaret ettiği nesnenin ömrünü RAII ile yönetir (Hafta 4); `Kutuphane` yıkıldığında `raf_` içindeki tüm materyaller — sanal yıkıcı sayesinde doğru türden — otomatik silinir.

Dikkat

`std::vector<Materyal>` yalnızca dilimlemekle kalmaz; `Materyal` ileride (Hafta 7) soyut yapıldığında *derlenmez* bile — soyut bir tip değerle örneklenemeyeceği

için vektör onu eleman olarak tutamaz. İşaretçiyle tutmak her iki sorunu birden çözer: gerçek türü korur *ve* soyut tabanlarla da çalışır. Çokbiçimli koleksiyonlarda taban işaretçisi (tercihen akıllı işaretçi) tutmak standart yaklaşımdır.

Bu ünitede kalıtım (is-a) ile kompozisyon (has-a) arasında "...dır" / "...e sahiptir" testiyle seçim yapmayı, kalıtımı kolay diye zorlamanın (vektörden türetme) neden kırılgan olduğunu, ve materyalleri tek bir kapta akıllı işaretçilerle çokbiçimli tutmayı öğrendiniz. Kitaplık teması artık kalıtım, kompozisyon ve çokbiçimlilikle organize; kurduğunuz `Materyal` iskeleti ve `Kutuphane` kabı, Hafta 7'de doğrudan büyütülecek başlangıç noktasıdır.

3 • Katılım — Şimdi Siz Deneyin

`Kutuphane` materyalleri `std::vector<Materyal>` ile mi yoksa `std::vector<std::unique_ptr<Materyal>>` ile mi tutmalı? İki gerekçe verin. Ayrıca `Kutuphane`'yi `std::vector`'dan *türetmek* iyi bir fikir mi?

Geribildirim. `std::vector<std::unique_ptr<Materyal>>` tercih edilir: (1) değerle tutmak materyalleri *dilimler* ve dinamik türü kaybeder, çokbiçimli `yazdir()` çalışmaz — işaretçi gerçek türü korur; (2) `unique_ptr` sahipliği RAII ile yönetir (Hafta 4), `Kutuphane` yıkılınca materyaller doğru türden otomatik silinir. `Kutuphane`'yi `std::vector`'dan türetmek yanlıştır: `Kutuphane` bir vektör *değildir* (is-a testi başarısız), bir vektöre *sahiptir* (has-a) — doğru seçim kompozisyondur.

Kendinizi Sınayın 6.4

`Kutuphane` materyalleri neden `std::vector<Materyal>` yerine `std::vector<std::unique_ptr<Materyal>>` ile tutar? En az iki gerekçe verin ve dilimleme ile sahiplik kavramlarına değinin.

5 • İzleme — Aktarım — Başka Nerede?

has-a / is-a kararı her tasarımda yeniden karşınıza çıkar ve "belirsizse kompozisyonu tercih et" öğüdü her yerde geçerlidir. `Kutuphane`'nin materyalleri tutan kabı, Hafta 9'da genel amaçlı bir `Liste<T>`'ye, Hafta 11'de bir arama ağacına evrilecek; ama has-a ilişkisi (kütüphane koleksiyona sahiptir) hep aynı kalacak. Çokbiçimli koleksiyon örüntüsü de — farklı türleri tek bir arayüz altında tek döngüde işlemek — her sistemde yeniden görülür: bir çizim programında farklı şekiller, bir oyun döngüsünde farklı nesneler, bir arayüzde farklı bileşenler hep bu örüntüyle yönetilir.

Bölüm Özeti

Özet

Geçen hafta tek bir değer tipini (`Vektor2B`) operatörleriyle donatıp `int` gibi doğal davranan tam bir tip oluşturdunuz; bu hafta odağınızı birden çok ilişkili tipi organize etmeye genişlettiniz. İlk ünitede kalıtımın neden gerektiğini gördünüz: ortak veriyi ve davranışı bir taban sınıfta (`Materyal`) toplayıp türetilmiş sınıflarla (`BasiliKitap`/`EKitap`/`SesliKitap`) özelleştirmek, "ortak olanı bir kez, farklı olanı bir kez" yazmayı sağlar. Kalıtımın doğru kullanıldığını "...dır" (is-a) testiyle belirlediniz, `: public Materyal` sözdizimiyle türetme yaptınız ve `public`/`private`'e üçüncü bir erişim düzeyi ekleyip `protected`'in kalıtım hattına açık, dışarıya kapalı olduğunu öğrendiniz. İkinci ünite nesnenin nasıl kurulduğuna odaklandı: türetilmiş kurucu, başlatma listesinde önce taban kurucusunu çağırır (kurucu zinciri); kurulumda taban önce, türetilmiş sonra oluşur, yıkımda tersi işler. Bir türetilmiş nesneyi taban türüne değerle kopyalamanın türetilmiş kısmı sessizce kestiğini — nesne dilimlemesi — ve bundan kaçınmak için işaretçi/referans kullanmak gerektiğini gördünüz. Üçüncü ünite çokbiçimliliği getirdi: `virtual` fonksiyonlar sayesinde bir taban işaretçisi üzerinden yapılan çağrı, nesnenin gerçek türüne göre çözülür (dinamik gönderim); `override` geçersiz kılmayı güvenli yapar; sanal yıkıcı taban işaretçiyle silmenin doğru yıkım zincirini işletmesini sağlar. Ayrıca saf sanal fonksiyon (`= 0`) ile soyut sınıf tanımlayıp — gündelik `Sekil` örneğiyle — bir tipin doğrudan örneklenmesini engellemeyi ve türetilmişlere bir sözleşme dayatmayı öğrendiniz. Dördüncü ünite tüm parçaları birleştirdi: kalıtım (is-a) ile kompozisyon (has-a) arasında "...dır"/"...e sahiptir" testiyle seçim yapmayı, `Kutuphane`'yi bir vektörden türetmenin neden yanlış (has-a, is-a değil) olduğunu ve materyalleri `std::vector<std::unique_ptr<Materyal>>` ile tek kapta çokbiçimli tutmayı gördünüz. Bu hafta `Materyal` tabanını somut tuttunuz ki dilimlemeyi gözlemleyebilesiniz; gelecek hafta `Materyal`'i soyut yapıp sanal bir `ozet()` ekleyerek bir `Materyal*` koleksiyonunu tam anlamıyla çokbiçimli işlemeyi (Çokbiçimlilik) derinleştireceksiniz.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — `protected` ile `private` farkı). Bir üyeyi `private` yerine `protected` yapmanın türetilmiş sınıflar açısından farkı, erişimdedir: `protected` bir üye, sınıfın kendisine *ve ondan türeyen sınıflara* açıktır; `private` bir üye ise *yalnızca* onu tanımlayan sınıfa açıktır, ondan türeyen sınıflar bile göremez. Bu yüzden `Materyal`'in `baslik_`, `yazar_`, `yil_` alanları `protected` yapılınca, `BasiliKitap`'ın kendi `yazdir()`'i bu alanlara hiçbir erişimci kullanmadan doğrudan ulaşabilir. Aynı alanlar `private` olsaydı türetilmiş sınıf onlara doğrudan erişemez, ancak tabanın bir erişimcisi (`baslik()`) üzerinden *okuyabilirdi*. Dışarıdan (sınıf dışından) erişim açısından ise `protected` ile `private` arasında *fark yoktur*: her ikisi de sınıf dışı koda kapalıdır — `bk.baslik_` her iki durumda da derleme hatasıdır. Kısacası `protected`, erişimi yalnızca kalıtım hattına genişletir; dışarıya karşı `private` kadar kapalı kalır.

Soru (Ünite 2 — kurucu/yıkıcı sırası ve dilimleme). Bir `BasiliKitap` oluşturulurken kurucular *taban önce, türetilmiş sonra* sırasıyla çalışır: önce `Materyal`

alt-nesnesi kurulur (izleme çıktısı `Materyal kuruldu`), sonra `BasiliKitap`'a özgü kısım (`BasiliKitap kuruldu`). Nesne yok edilirken sıra tam tersidir: *türetilmiş önce, taban sonra* — önce `BasiliKitap yikildi`, sonra `Materyal yikildi`. Bunun mantığı, türetilmişin tabanın üzerine oturmasıdır: altındaki taban önce var olmalı ve en son yıkılmalıdır (bina temeli gibi). `Materyal m = bk;` satırında ise nesne dilimlemesi olur: `m` bir `Materyal` *değeri* olduğundan yalnızca bir taban kadar yer tutar; `Materyal`'in kopya kurucusu `bk`'nin yalnızca taban alt-nesnesini (`baslik_`, `yazar_`, `yil_`) kopyalar, `sayfaSayisi_` ve türetilmişe özgü davranış kesilir. Bu yüzden `m.yazdir()`, `BasiliKitap`'ın değil `Materyal`'in sürümünü çağırır ve çıktıda sayfa sayısı görünmez.

Soru (Ünite 3 — sanal olmayan yıkıcıyla taban işaretçiyle silme). `~Materyal()` `virtual` *değilken* bir `Materyal* p` bir `BasiliKitap`'a işaret ederken `delete p;` yaparsanız, bu çağrı yalnızca taban yıkıcısını (`~Materyal`) çağırır; türetilmişin yıkıcısı (`~BasiliKitap`) hiç çalışmaz. Bunun nedeni, yıkıcı sanal olmadığında çağrının `p`'nin *bildirilen* türüne (`Materyal`) sabitlenmesi, gerçek türüne (`BasiliKitap`) bakılmamasıdır. Sonuç olarak türetilmiş sınıfın ayırdığı ek kaynaklar temizlenmeden kalır — bu tanımsız davranıştır ve kaynak sızıntısına yol açabilir. Doğru davranışı sağlamak için tabanın yıkıcısı `virtual` yapılmalıdır (`virtual ~Materyal();`); o zaman `delete p` önce `~BasiliKitap`'ı, sonra `~Materyal`'i sırayla çağırır ve nesne eksiksiz temizlenir. Genel kural: taban işaretçisiyle silinebilecek her polimorfik tabanın yıkıcısı `virtual` olmalıdır.

Soru (Ünite 4 — neden `unique_ptr` vektörü). Materyalleri değerle değil akıllı işaretçiyle tutmanın iki ciddi gerekçesi vardır ve ikisi de bu haftadan gelir. Birincisi dilimlemedir: bir `std::vector<Materyal>` her elemanı bir `Materyal` *değeri* olarak saklar; bir `BasiliKitap` eklendiğinde türetilmiş kısmı kesilir, dinamik türü kaybolur ve `yazdir()` artık taban sürümünü çağırır — çokbiçimlilik bozulur. İşaretçi ise nesneyi kopyalamaz, gerçek türünü olduğu gibi korur; böylece `m->yazdir()` her materyal için doğru (gerçek türe ait) sürümü çağırır. İkincisi sahipliktir: `std::unique_ptr`, işaret ettiği nesnenin ömrünü RAII ile yönetir (Hafta 4); `Kutuphane` yıkıldığında `raf_` içindeki tüm materyaller — sanal yıkıcı sayesinde doğru türden — otomatik silinir. Ek olarak, `Materyal` ileride soyut yapıldığında `std::vector<Materyal>` derlenmez bile (soyut tip değerle örneklenemez), oysa işaretçi kabı soyut tabanlarla da çalışır. Bu yüzden çokbiçimli koleksiyonlarda taban akıllı işaretçisi tutmak standart yaklaşımdır.